Anti-Patterns Analysis - Original Parking Lot Manager

Executive Summary

The original Parking Lot Manager app exhibits 10 anti-patterns that negatively impact code quality, maintainability, and scalability. Here, we identify these issues and explain why they are problematic.

1. God Class Anti-Pattern

Location: `ParkingLot` class in `ParkingManager.py`

Problem: The ParkingLot class contains more than 40 methods handling:

- Business logic (parking, leaving, searching)
- Data persistence (slots management)
- UI interaction (direct tkinter calls)
- Validation
- Fee calculation

Impact:

- Violates Single Responsibility Principle
- Changes in UI affect business logic
- Cannot reuse business logic without UI
- Impossible to test individual concerns
- Difficult to maintain and extend

Evidence: Lines 20-295 in ParkingManager.py show one class doing everything.

2. Tight Coupling with UI Framework

Location: Throughout `ParkingLot` class

Problem: Business logic methods directly manipulate tkinter widgets:

```
def makeLot(self):
    tfield.insert(tk.INSERT, output)  # Direct UI dependency
```

Impact:

- Cannot test business logic without GUI
- Cannot swap UI frameworks
- Business logic tied to presentation

Evidence: Lines 125, 142, 161, 172, 193, 210, 221, 236, 257, 275

3. Global Variables

Location: Module level in `ParkingManager.py`

Problem: More than 15 global variables for UI state:

```
command_value = tk.StringVar()
num_value = tk.StringVar()
ev_value = tk.StringVar()
# ... 12 more
```

Impact:

- Hidden dependencies
- Makes testing impossible
- Violates encapsulation

Evidence: Lines 12-26

4. Broken Inheritance Hierarchy

Location: `ElectricVehicle.py`

Problem: ElectricVehicle does NOT extend Vehicle class:

```
class ElectricVehicle:  # Should extend Vehicle
    def __init__(self, regnum, make, model, color):
        # Duplicates all Vehicle attributes
```

Impact:

- Code duplication (4 identical attributes)
- Parallel class hierarchies (Vehicle vs ElectricVehicle)
- Cannot treat all vehicles polymorphically
- ElectricCar/ElectricBike use wrong inheritance pattern

Evidence: ElectricVehicle.py lines 1-24

5. Improper Use of `__init__` in Subclasses

Location: `Vehicle.py`, `ElectricVehicle.py`

Problem: Subclasses call parent `__init__` directly instead of using `super()`:

```
class Car(Vehicle):
    def __init__(self, regnum, make, model, color):
        Vehicle.__init__(self, regnum, make, model, color)  # Wrong
```

Should be:

```
super().__init__(regnum, make, model, color)
```

Impact:

- Violates Python best practices
- Breaks with multiple inheritance
- Makes refactoring harder

Evidence: Vehicle.py lines 22, 29, 36, 43 and ElectricVehicle.py lines 26, 32

6. No Data Access Abstraction

Location: `ParkingLot` class

Problem: Business logic directly manipulates arrays:

```
self.slots = [-1] * capacity
self.slots[slotid] = vehicle
if self.slots[i] == -1:
```

Impact:

- Cannot switch storage (database, file, cloud)
- No transaction support
- No data validation layer
- Tight coupling to in-memory arrays

Evidence: Throughout ParkingLot class

7. Mixed Return Types

Location: `park()` method in `ParkingLot`

Problem: Returns different types based on conditions:

```
def park(self, ...):
    if success:
        return slotid  # Returns integer
    else:
        return -1      # Returns error code
```

Impact:

- Caller must check type
- No clear success/failure indication
- -1 could be valid slot in some systems

- Error handling is implicit

Evidence: ParkingManager.py lines 57-81

8. Inconsistent Naming Conventions

Location: Throughout codebase

Problem: Mix of naming styles:

- camelCase: `getEmptySlot`, `slotid`, `regnum`
- snake_case: `command_value`, `num_value`
- abbreviations: `ev`, `reg`

Impact:

- Reduces code readability
- Makes code harder to understand

Evidence: Throughout all files

9. Long Method Bodies

Location: `park()`, `status()`, `chargeStatus()` methods

Problem: Methods exceed 30-50 lines with deeply nested logic:

```
def park(self, regnum, make, model, color, ev, motor):
    if (self.numOfOccupiedEvSlots < self.evCapacity or ...):
        slotid = -1
        if (ev == 1):
            if self.numOfOccupiedEvSlots < self.evCapacity:
                # ... 15 more lines of nested logic
```

Impact:

- Difficult to understand
- Hard to test individual conditions
- High complexity
- Error-prone

Evidence: ParkingManager.py lines 57-81, 91-115

10. No Error Handling

Location: Throughout codebase

Problem: No try-catch blocks, no input validation, no error messages:

```
def createParkingLot(self, capacity, evcapacity, level):
    self.slots = [-1] * capacity  # What if capacity < 0?
```

Impact:

- Crashes on invalid input
- No user feedback on errors
- Difficult to debug

Evidence: No try-except blocks anywhere in codebase

Remediation Approach

The refactored design addresses these issues through:

1. **MVP Pattern** - Separates presentation from business logic
2. **Repository Pattern** - Abstracts data access
3. **Factory Pattern** - Centralizes object creation
4. **Proper Inheritance** - Fixes vehicle class hierarchy
5. **Separation of Concerns** - Each class has one responsibility
6. **Dependency Injection** - Removes global state
7. **Consistent Naming** - Follows PEP 8
8. **Error Handling** - Adds validation and exception handling

Please see `design_patterns_justification.pdf` for more on remediation strategy.